

Database Indexing

Local Host (IP Address) $\xrightarrow{\text{API call}}$ Backend $\rightarrow$ Database



Analogy

Home Restaurant (Address) $\xrightarrow{\text{order}}$ Chef $\rightarrow$ Fridge with ingredients

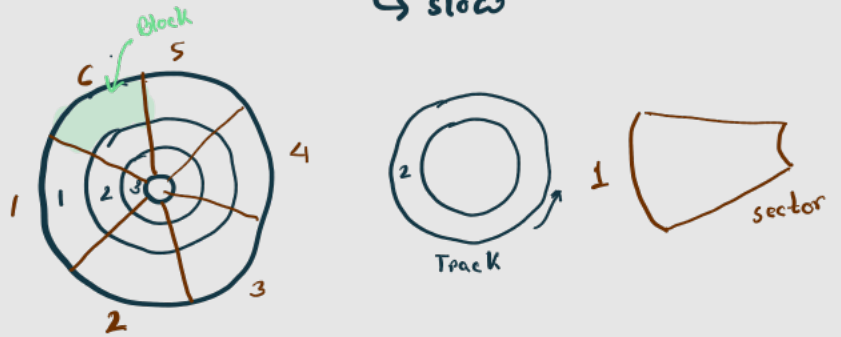
CAB

id	name	dept	cg	section
✓	✓	✓	✓	✓

student

Hard disc/SSD [non-volatile storage]

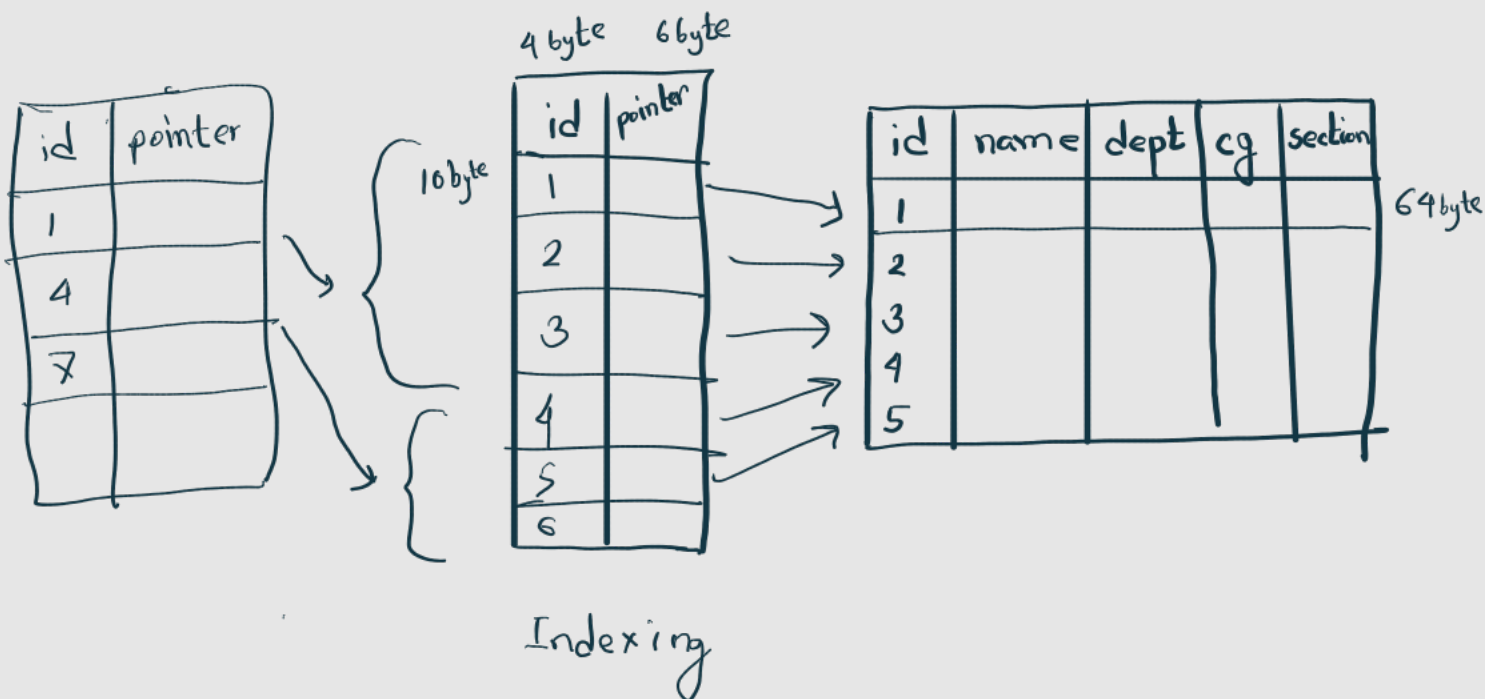
$\hookrightarrow$ slow



Track - 1, Sector - 6

RAM [volatile]

SELECT name FROM student WHERE id > 4



Multi level indexing

Primary / Clustering index

- (1) Physical data on the disk **sorted** using this key
- (2) Only 1 per table
- (3) Instantly retrieve the record

Secondary / Non clustering index

- (1) Not sorted by this key
- (2) Multiple
- (3) Retrieve info using another column by creating a separate index that points to the row in main table.

Dense index file

- (1) Entry for every record
- (2) Fast
- (3) More memory
- (4) For both primary and secondary index

Sparse index file

- (1) Entry for some record
- (2) Slow
- (3) Less memory
- (4) For primary index

10101	→	10101	Srinivasan	Comp. Sci.	65000	→
12121	→	12121	Wu	Finance	90000	→
15151	→	15151	Mozart	Music	40000	→
22222	→	22222	Einstein	Physics	95000	→
32343	→	32343	El Said	History	60000	→
33456	→	33456	Gold	Physics	87000	→
45565	→	45565	Katz	Comp. Sci.	75000	→
58583	→	58583	Califieri	History	62000	→
76543	→	76543	Singh	Finance	80000	→
76766	→	76766	Crick	Biology	72000	→
83821	→	83821	Brandt	Comp. Sci.	92000	→
98345	→	98345	Kim	Elec. Eng.	80000	→

Dense

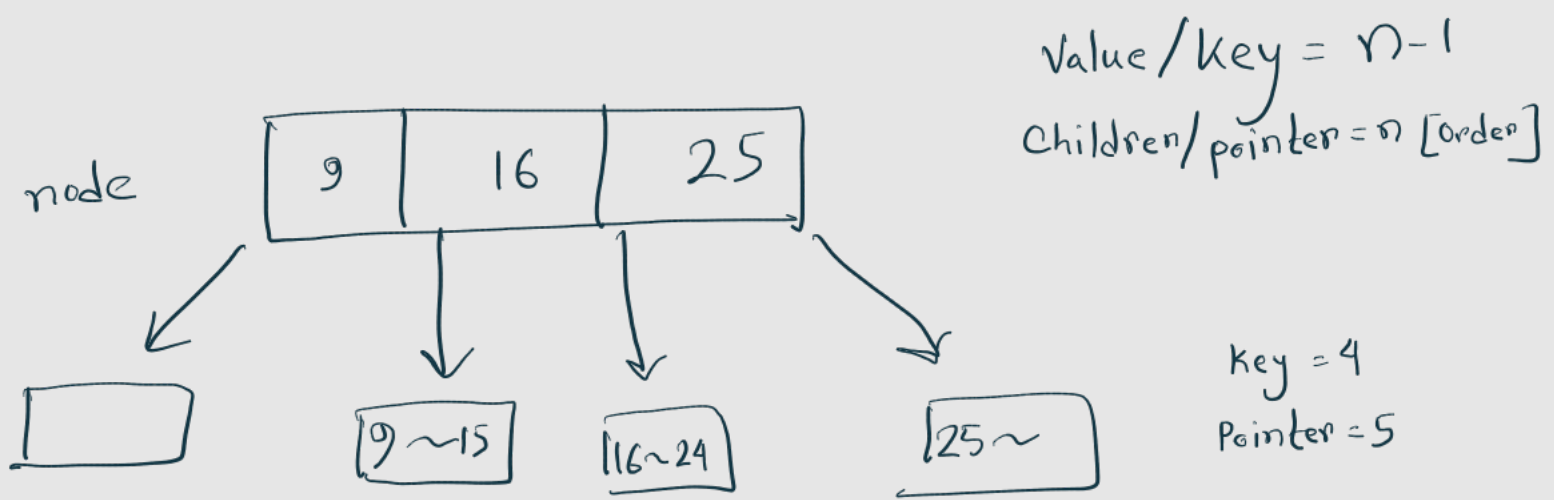
sort (Primary)
↓

Biology	→	76766	Crick	Biology	72000	→
		10101	Srinivasan	Comp. Sci.	65000	→
		45565	Katz	Comp. Sci.	75000	→
		83821	Brandt	Comp. Sci.	92000	→
Finance	→	98345	Kim	Elec. Eng.	80000	→
		12121	Wu	Finance	90000	→
		76543	Singh	Finance	80000	→
		32343	El Said	History	60000	→
		58583	Califieri	History	62000	→
		15151	Mozart	Music	40000	→
		22222	Einstein	Physics	95000	→
Physics	→	33465	Gold	Physics	87000	→

Sparse

Figure 2

B⁺ Tree



B⁺ order = 4 ; key = 3
↓
pointer

Key size = 4 bit, child 6 bit. Find block size/node

$$\begin{aligned} \text{Node size/Block} &= \text{Key} + \text{Pointer size} \\ &= 4(n-1) + 6n \quad [\text{Let pointer} = n] \end{aligned}$$

key/value

$$\frac{\min}{\left\lceil \frac{n}{2} \right\rceil - 1}$$

$$\frac{\max}{n-1}$$



Pointer/Child/
order

$$\left\lceil \frac{n}{2} \right\rceil$$

$$n$$

$$\lceil 1.2 \rceil = 2 \quad [\text{Ceil}]$$

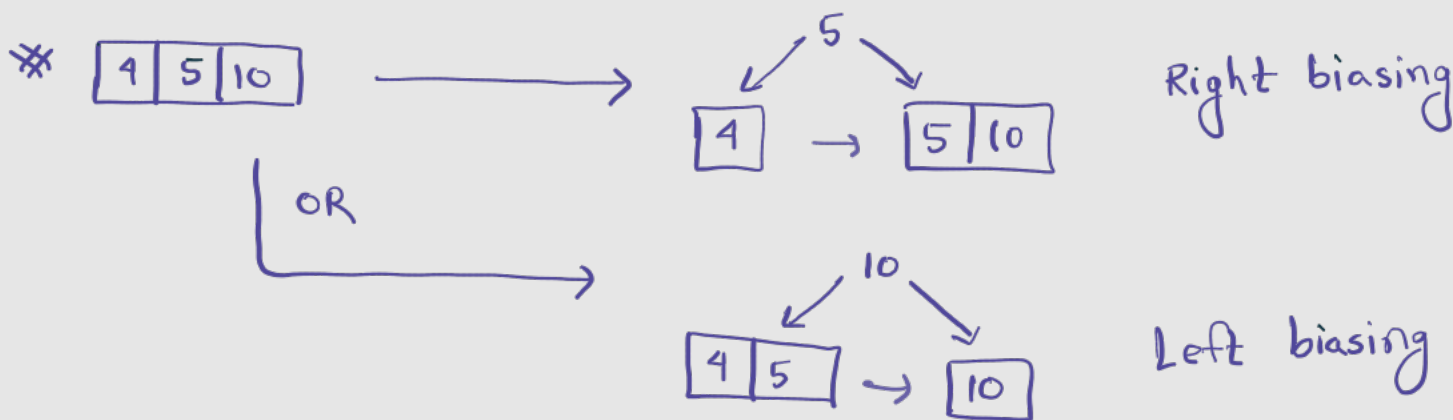
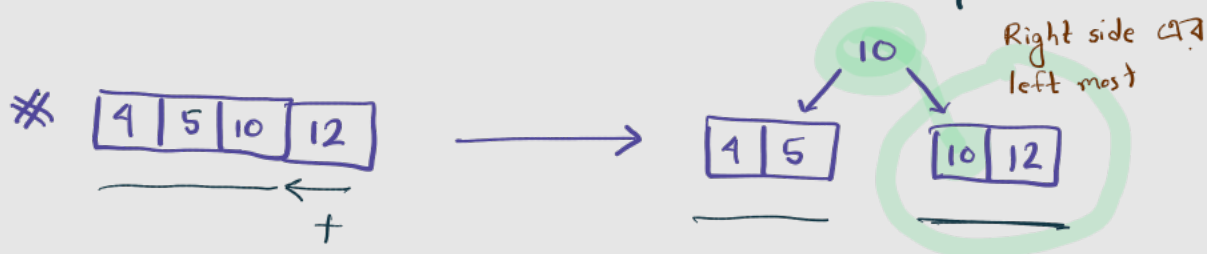
$$\lfloor 1.9 \rfloor = 1 \quad [\text{Floor}]$$

★ Insertion in B⁺ Tree:

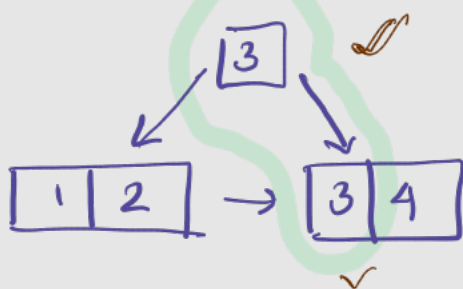
Rules:

(1) Maximum child এর বেশি হলে split.

key = 3
order = 4

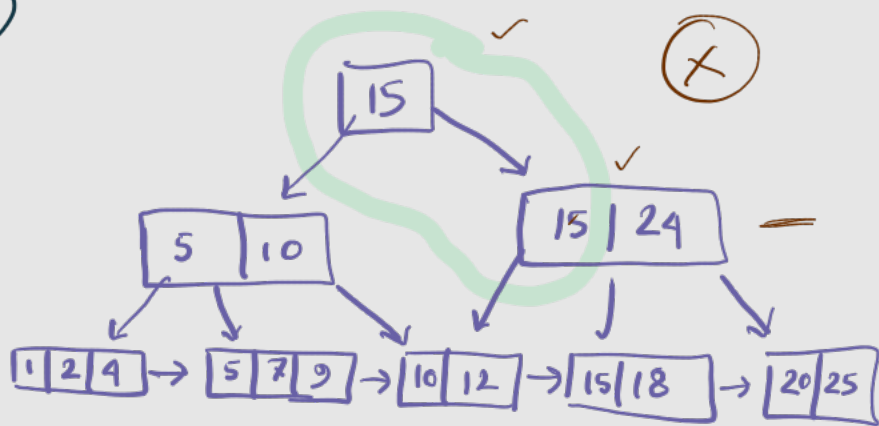


(2)

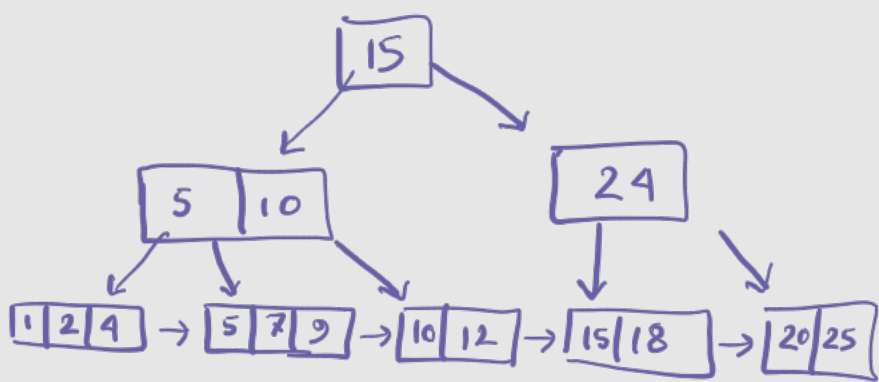


* Leaf can have repeats

(3)

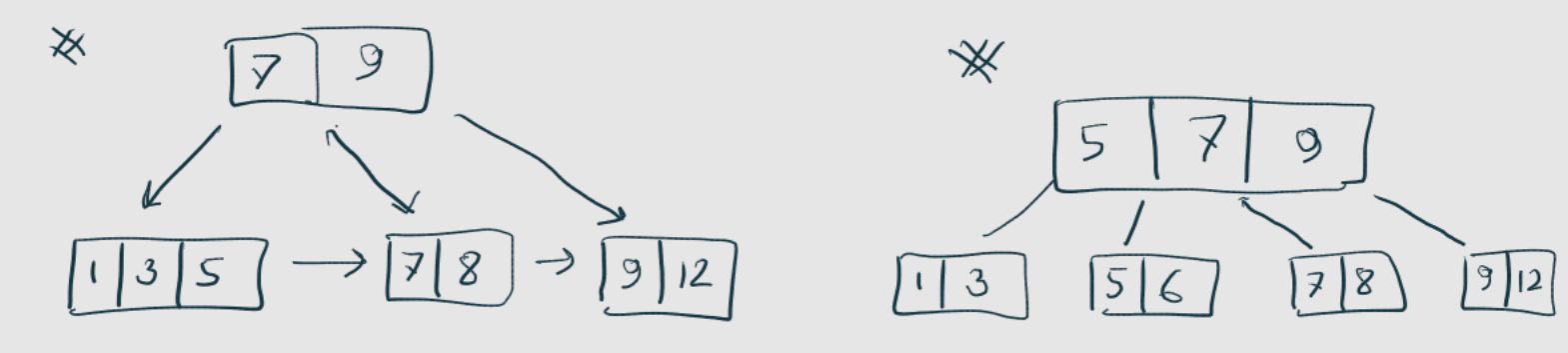
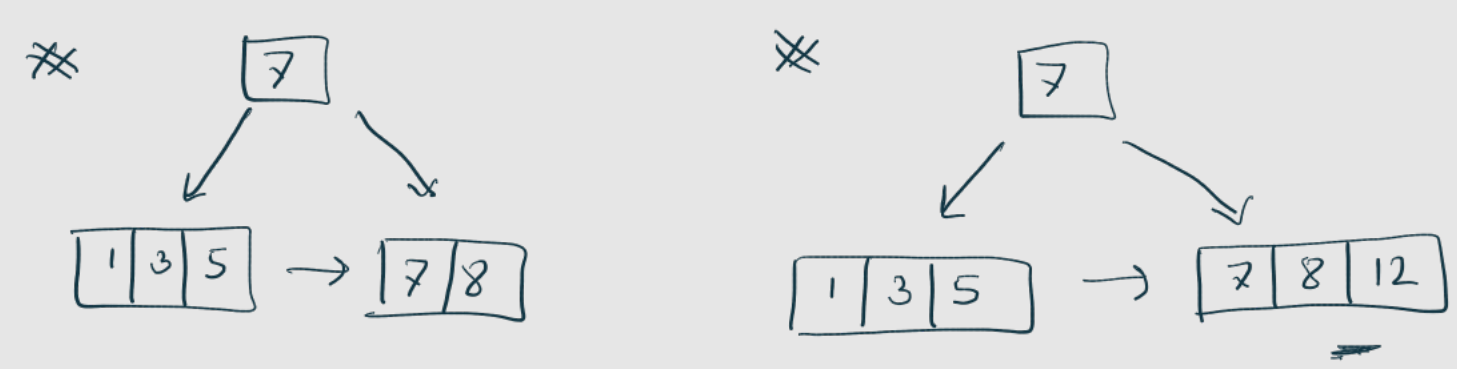


* Internal node can not have repeats



Insert : ~~8~~ ~~5~~ ~~1~~ ~~7~~ ~~3~~ ~~12~~ ~~9~~ ~~6~~ in B⁺ tree
order 4

	<u>Min</u>	<u>Max</u>
Key	-	3
Pointer	-	4



 (a) Insert : ~~0~~ ~~5~~ ~~1~~ ~~7~~ ~~3~~ ~~12~~ ~~9~~ ~~8~~ ~~13~~ in B⁺ tree
 order 3

(b) Find 6

	<u>min</u>	<u>max</u>
key		2
Pointer		3

